



National University
Of Computer and Emerging Sciences

Software Re-Engineering

Assignment # 1

Group Members:

- | | |
|--------------------------|----------|
| 1. Kisaa Batool | 20i-1829 |
| 2. Umar Waheed | 20i-0926 |
| 3. Maaidah Arif Siddiqui | 20i-0997 |
| 4. Aleena Fatima Khalid | 20k-1688 |

Table of Contents

1. Project Overview.....	2
2. Project Features.....	2
3. Analysis Techniques Used.....	2
4. Strengths and Weaknesses.....	2
5. Areas for improvement.....	2
6. Mental Model.....	2
7. Abstract Syntax Tree.....	2

1. Project Overview

Name: 20 Questions Game

Description: The "20 Questions Game" is an interactive text-based guessing game implemented in Java. It allows users to play a game where they think of an object, and the computer attempts to guess that object by asking up to 20 yes-or-no questions. The game provides a fun and engaging experience as players interact with the computer to refine its guesses through user-provided questions and answers. The project aims to create an entertaining and educational gaming experience while showcasing key programming concepts such as binary trees and recursion.

2. Project Features

- **User-Friendly Interface:** The game offers a straightforward text-based user interface that guides players through each round of the game.
- **Interactive Gameplay:** Players can engage in a game of wit with the computer by answering questions and attempting to stump it with their chosen object or word.
- **Learning and Adaptation:** The computer becomes smarter over time by learning from player-contributed questions and answers. This adaptive learning mechanism adds depth to the game.
- **Save and Load Games:** Players have the option to save their current game progress to a file for later retrieval, allowing for continued play and game preservation.
- **Statistics Tracking:** The game keeps track of important statistics, including the total number of games played and the number of games won by the computer. This feature adds a competitive element to the game.
- **File I/O:** The game utilizes file input and output to load and save game data, enhancing its persistence and replayability.

- **Error Handling:** The code incorporates error handling to ensure that the game can gracefully handle exceptions, such as missing files or invalid input.
- **Tree Data Structure:** The project demonstrates the use of binary trees to organize and manage the game's question-answer pairs efficiently.
- **Modular Code:** The code is well-structured and modular, making it easy to understand, maintain, and extend.
- **Recursion:** Recursion is used in various parts of the project, including the game's main loop and the loading of question trees from files.
- **Clear User Prompts:** The game provides clear prompts to guide players in their interactions with the computer, ensuring a smooth and engaging experience.
- **Persistence:** The ability to save and load games ensures that players can continue their gaming sessions across multiple sessions.

3. Analysis Techniques Used

Risk Assessment and Mitigation

- **Bug and Software Defects:**
 - Risk: The game may contain software bugs or defects that could lead to unexpected behavior or crashes.
 - Impact: User experience would be negatively affected, and players may lose interest.
 - Probability: Moderate, as software can have unexpected issues.
 - Mitigation: Regular testing and quality assurance procedures should be implemented to identify and fix bugs. Beta testing with users can help uncover issues.
- **Limited User Engagement:**
 - Risk: Players might not find the game engaging enough, leading to low usage or abandonment.
 - Impact: The project may not achieve its intended goals if users lose interest quickly.
 - Probability: Moderate, as user engagement can be unpredictable.

- Mitigation: Conduct user surveys or interviews to gather feedback on gameplay. Use this feedback to make improvements and enhance engagement.
- **Obsolete Technology Stack:**
 - Risk: The game may rely on outdated technology or libraries that are no longer supported or secure.
 - Impact: Security vulnerabilities could arise, and it may be challenging to find developers familiar with outdated technology.
 - Probability: Low to moderate, depending on the age of the codebase.
 - Mitigation: Consider migrating the codebase to a more modern and well-supported technology stack. Keep libraries and dependencies up to date.

User Experience (UX) Assessment:

1. User Interface (UI) Assessment:

- The "20 Questions Game" primarily relies on a text-based interface, which can be effective for its genre. However, there are aspects to consider:
 - Positives:
 - The text-based interface is simple and accessible, making it easy for users to understand and play.
 - The prompts and questions presented to the user are clear and concise.
 - Areas for Improvement:
 - Enhance the visual presentation of the game with ASCII art or simple graphics to make it more visually appealing.
 - Consider using different text formatting or colors to differentiate between user prompts and computer responses.
 - Provide a welcome message or game instructions at the start to orient new users.

2. Gameplay Experience:

- The core gameplay revolves around the computer's ability to guess the user's object through a series of yes/no questions. Here's an assessment:
 - Positives:
 - The game mechanics are straightforward and easy to grasp.
 - The recursive nature of the game adds depth and engagement.

- Areas for Improvement:
 - Consider adding some variety to the questions asked by the computer to make the game more interesting and less predictable.
 - Provide a scoring system or feedback mechanism to incentivize players to continue playing and improving.

3. User Guidance and Feedback:

- Providing clear guidance and feedback is crucial for user engagement and understanding.
 - Positives:
 - The game asks questions in a clear and concise manner.
 - User responses are processed effectively.
 - Areas for Improvement:
 - Add more descriptive feedback for user inputs, especially when they contribute a new question or when the computer wins or loses.
 - Include hints or tips to assist users who may be unfamiliar with the "20 Questions" game concept.

4. Engagement and Retention:

- Keeping users engaged and encouraging them to return to the game is essential.
 - Positives:
 - The concept of the computer learning from user input adds an element of long-term engagement.
 - Areas for Improvement:
 - Implement a scoring system or achievements to motivate users to play multiple rounds.
 - Consider adding a leaderboard to foster competition among players.

5. Accessibility and Compatibility:

- Ensure that the game is accessible to a wide range of users and compatible with various devices and browsers.
 - Positives:
 - Text-based games are inherently accessible to most users.
 - Areas for Improvement:

- Test the game on various platforms and browsers to ensure compatibility and responsiveness.
- Provide keyboard shortcuts or alternative input methods for users with disabilities.

6. Overall User Experience:

- The overall user experience is the result of the cumulative impact of all elements mentioned above.
 - Positives:
 - The game is functional and can provide an enjoyable experience.
 - Areas for Improvement:
 - Continuously gather user feedback to make iterative improvements to the UI, gameplay, and overall experience.
 - Consider conducting usability testing to identify specific pain points and areas for enhancement.

Code Review and Analysis

1. Code Structure and Organization:

- File Structure: The code is organized into multiple Java files, which is good for modularity and maintainability.
- Class Hierarchy: The code consists of three primary classes: `UserInterface`, `QuestionTree`, and `QuestionNode`. This separation of concerns is appropriate for a relatively simple game.
- Method Naming: Method names are descriptive and follow standard Java naming conventions, making the code easy to understand.
- Comments: The code includes comments explaining the purpose and functionality of various methods, which aids in code comprehension.

2. Coding Practices:

- Recursion Usage: The use of recursion in the `QuestionTree` class to traverse the game tree is appropriate and follows the game's logic.
- Error Handling: The code handles potential exceptions, such as null inputs, by throwing `IllegalArgumentException`s, which is a good practice.

3. Code Smells and Potential Weaknesses:

- UserInterface Class: The UserInterface class contains a single static method for reading user input. While this is functional, it could be encapsulated within a more comprehensive user interface class if the game were to expand with additional user interactions.
- QuestionNode Class: The QuestionNode class appears to serve multiple roles, including representing both questions and answers. It could benefit from a clearer separation of responsibilities.
- String Manipulation: The code frequently manipulates strings to extract and format data. While this is common in text-based applications, it's essential to ensure that string manipulation is efficient and free from potential errors.

4. Outdated Elements:

- Language Version: The code does not indicate the specific Java version it targets. While this may not be an issue now, it's essential to specify the Java version explicitly to ensure compatibility and take advantage of language improvements.
- Libraries: The code does not rely on external libraries, which is a positive aspect for simplicity and ease of maintenance.

5. Code Duplication:

- Repetitive Code: Some code blocks appear more than once, such as the logic for reading user input in the UserInterface class. Consider centralizing such code to avoid redundancy.

Overall Code Assessment: The code for the "20 Questions Game" is well-structured, follows coding conventions, and is relatively clean. While there are opportunities for improvement, such as refining the role of the QuestionNode class and addressing potential code duplication, the code effectively implements the game's logic.

Functional Analysis

Current Functionality:

1. Game Initialization:
 - The game starts by initializing a binary tree with a predefined root question: "Would your object happen to be Jedi?"
2. Gameplay Flow:

- The game follows a recursive flow:
 - It asks questions about the user's object.
 - Based on the user's answers (yes or no), it traverses the binary tree.
 - It continues asking questions and making guesses until it either correctly guesses the object or runs out of questions.
 - If the computer guesses correctly, it wins; otherwise, the user provides a new question and answer to improve the game's intelligence.
- 3. User Interaction:
 - The game relies on the `UserInterface` class to read user input for yes/no questions and user responses to the computer's guesses.
 - Users can start new games, continue previous games by providing a file name, and choose whether to store the current game data in a new file.
- 4. Game Statistics:
 - The system keeps track of the number of games played and the number of games won by the computer.
- 5. Saving and Loading Games:
 - Users can save the current game state to a file and later load it to continue playing from where they left off.

Alignment with Business Requirements:

It's important to note that the "20 Questions Game" is primarily an educational and entertainment program rather than a business application. Therefore, it doesn't have direct business requirements in the traditional sense. However, it aligns with the following potential goals:

- User Engagement: The game aims to engage users in a fun and interactive guessing game, making it suitable for educational or entertainment purposes.
- Learning and Adaptation: The game encourages users to contribute new questions, which can enhance the computer's ability to guess objects accurately over time. This adaptive learning mechanism adds depth to the game.

Obsolete or Redundant Functionality:

While the core functionality of the game aligns with its purpose, there are areas where functionality might be considered obsolete or redundant:

- User Interface Simplification: The game could benefit from a more user-friendly and visually appealing interface. While the text-based interface is functional, it may be considered basic by modern standards.
- Limited Gameplay Variety: The game follows a fixed pattern of asking yes/no questions, which might become predictable after multiple plays. Adding variability to the questions or introducing additional game modes could enhance the user experience.
- Lack of Scoring System: The game lacks a scoring system or rewards for user achievements, which could further engage and motivate players.

4. Strengths and Weaknesses

Strengths:

- Educational Value: The game serves as an educational tool that encourages users to think logically and contribute questions to improve the computer's guessing ability. It can be used for learning and fun.
- Simple and Understandable: The code is relatively simple and easy to understand, making it accessible for developers, educators, and students interested in programming and decision tree algorithms.
- Recursive Algorithm: The use of recursion to traverse the decision tree is a strong point, as it demonstrates a fundamental programming concept in a practical context.
- Adaptive Learning: The game's adaptive learning mechanism, where the computer becomes smarter with user-contributed questions, adds depth and engagement to the gameplay.
- File Handling: The ability to save and load game data to and from files allows users to continue their games, enhancing the user experience.

Weaknesses:

- Limited User Interface: The text-based user interface may not be engaging for all users, and it lacks modern UI/UX features that could make the game more appealing.
- Predictable Gameplay: The game follows a fixed pattern of asking yes/no questions, which may become predictable after multiple plays. Variability in question types or additional game modes could enhance replayability.

- Input Validation: The code lacks robust input validation, which could lead to unexpected behavior or vulnerabilities if users provide malicious input.
- Security Concerns: While not a critical issue for this type of game, the code could benefit from improved security practices, especially related to input validation and file handling.
- Scalability: The code is designed for single-player use and may not easily scale to accommodate multiplayer or more complex game scenarios without significant modifications.
- Minimal Documentation: The code could benefit from more comprehensive documentation, including inline comments and high-level explanations of key components and algorithms.
- Code Refactoring: There are opportunities for code refactoring to enhance readability and maintainability, especially in the QuestionTree class, which handles most of the game logic.

5. Areas for improvement

1. User Interface Enhancement:

- Issue: The current text-based user interface is basic and lacks modern UI/UX features.
- Improvement: Consider enhancing the user interface with graphics, animations, or a web-based interface to make the game more visually appealing and engaging.

2. Gameplay Variety:

- Issue: The game follows a predictable pattern of asking yes/no questions, which may become monotonous for repeated plays.
- Improvement: Introduce variability in question types, expand the game's question database, or add different game modes to enhance replayability.

3. Input Validation:

- Issue: The code lacks robust input validation, which could lead to unexpected behavior or vulnerabilities.
- Improvement: Implement thorough input validation to handle unexpected or malicious user input gracefully. Validate user responses and file names to prevent errors or security risks.

4. Security Practices:

- Issue: While not a critical concern for this type of game, improved security practices can be beneficial.

- Improvement: Apply security best practices, especially related to input validation, to prevent potential vulnerabilities. Implement secure file handling to avoid unauthorized access or manipulation.

5. Scalability Consideration:

- Issue: The code is designed for single-player use and may not easily accommodate multiplayer or more complex game scenarios.
- Improvement: If considering expansion, plan for scalability and consider how the code architecture would adapt to support additional players, game modes, or features.

6. Documentation Enhancement:

- Issue: The code could benefit from more comprehensive documentation.
- Improvement: Add inline comments to explain complex logic or decision-making processes. Provide high-level explanations of key components, algorithms, and classes to assist future developers in understanding the codebase.

7. Code Refactoring:

- Issue: Some parts of the code, particularly in the QuestionTree class, may benefit from refactoring for improved readability and maintainability.
- Improvement: Consider refactoring the code to follow best practices, adhere to coding conventions, and organize the code more efficiently.

8. Modernization of User Interface Libraries:

- Issue: If the game interface is updated, ensure that any libraries or frameworks used for UI are up to date.
- Improvement: If the user interface is modernized, make sure to use current libraries and technologies for the best user experience.

6. Mental Model

1. Main Program (QuestionMain)

- Represents the entry point of the program.
 - Controls the flow of the game and user interactions.
 - Manages game data loading, playing, and saving.
 - Utilizes the **QuestionTree** and **UserInterface** classes.
2. **User Interface (UserInterface)**
 - Provides a simple interface for user interactions via the console.
 - Contains the **nextAnswer** method for reading user input.
 3. **Question Tree (QuestionTree)**
 - Represents the core logic of the game.
 - Manages the binary tree of questions and answers.
 - Handles game initialization, playing, saving, and loading.
 - Utilizes the **QuestionNode** class to build the tree.
 4. **Question Node (QuestionNode)**
 - Represents nodes within the binary tree.
 - Can be either a question or an answer.
 - Contains references to left and right child nodes.
 - Stores the content of the question or answer.

Game Flow:

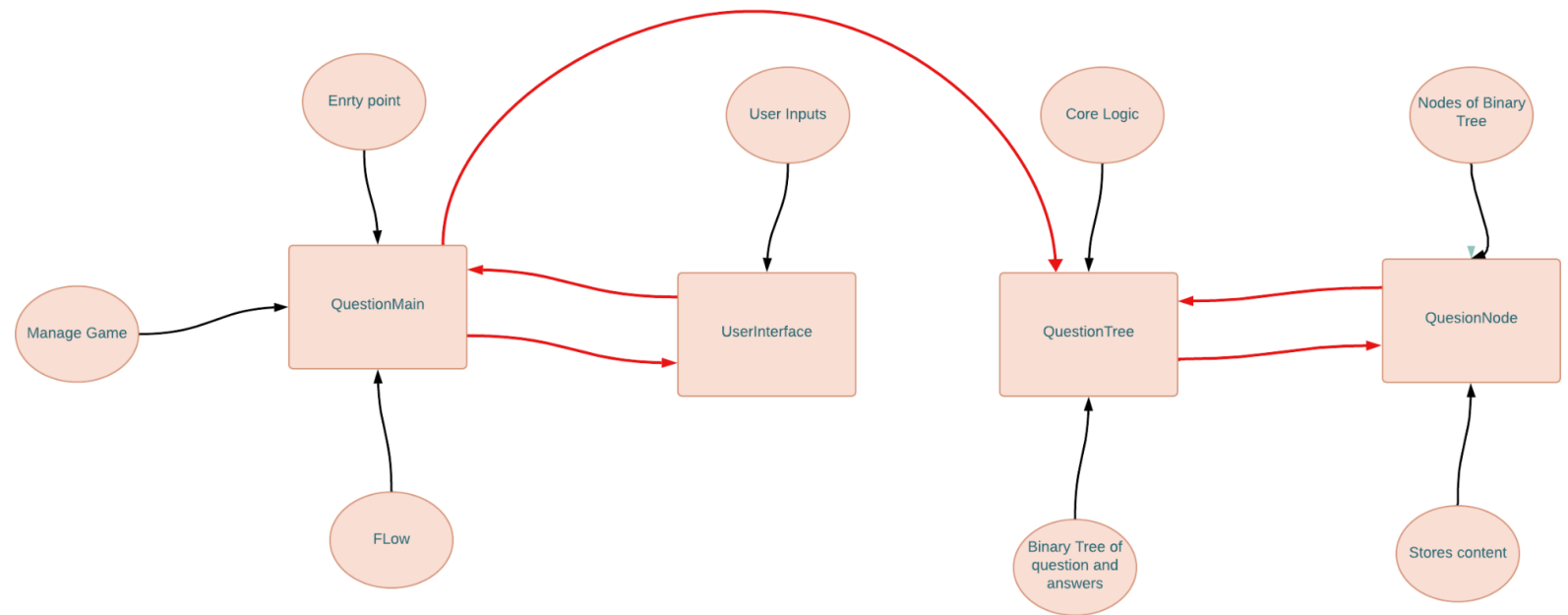
- The program starts in the **QuestionMain** class.
- It loads existing game data if available.
- It enters a main game loop.
- The user plays the game by answering questions.
- The **QuestionTree** manages the game logic, asks questions, and handles user responses.
- The game can be replayed, and data can be saved for future sessions.

Key Interactions:

- The user interacts with the console through the **UserInterface**.
- The **QuestionMain** orchestrates the game flow.
- The **QuestionTree** handles game-specific logic and tree management.
- The **QuestionNode** class represents nodes in the binary tree.

Game Logic:

- The game uses a binary tree to guess an object the user is thinking of.
- Questions are asked to narrow down the possibilities.
- If the computer guesses correctly, it wins; otherwise, it learns from the user's input and adapts for the next game.



7. Abstract Syntax Tree

```

/**
 * A basic text user interface for the 20 questions game.
 */
public class QuestionMain {

    private static final String PLAY_AGAIN_MESSAGE = "Challenge me again? ";

    private static final String SAVE_MESSAGE = "Shall I remember these games? ";

    private static final String LOAD_MESSAGE = "Shall I recall our previous games? ";

    private static final String SAVE_LOAD_FILENAME_MESSAGE = "What is the file name? ";

    private static final String STATUS_MESSAGE = "Games played: %d\nI won: %d";

    private static final String BANNER_MESSAGE = "Think of an item, and I will guess it.";

    public static void main(String[] args) {
        QuestionMain tq = new QuestionMain();
    }
}

```

```

        tq.run();
    }

    // fields
    private Scanner input = new Scanner(System.in);

    private PrintStream output = System.out;

    private QuestionTree tree;

    /**
     * Constructs a text user interface and its question tree.
     */
    public QuestionMain() {
        tree = new QuestionTree(input, output);
    }

    // private helper for overall game(s) loop
    private void run() {
        output.println("Welcome to the game of 20 Questions!");
        load();
        // "Think of an item, and I will guess it in N tries."
        output.println("\n" + BANNER_MESSAGE);
        do {
            // play one complete game
            // blank line between games
            output.println();
            tree.play();
            output.print(PLAY_AGAIN_MESSAGE);
        } while (!promptToPlayAgain());
        // print overall stats
        // Games played: N ... I have won: M
        output.println("\n" + String.format(STATUS_MESSAGE, tree.totalGames(),
        tree.gamesWon()));
        save();
    }

    // common code for asking the user whether they want to save or load
    private void load() {
        output.print(LOAD_MESSAGE);
        if (UserInterface.nextAnswer(input)) {
            output.print(SAVE_LOAD_FILENAME_MESSAGE);
            String filename = input.nextLine();

```

```

        try {
            Scanner in = new Scanner(new File(filename));
            tree.load(in);
        } catch (FileNotFoundException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}

// common code for asking the user whether they want to save or load
private void save() {
    output.print(SAVE_MESSAGE);
    if (UserInterface.nextAnswer(input)) {
        output.print(SAVE_LOAD_FILENAME_MESSAGE);
        String filename = input.nextLine();
        try {
            PrintStream out = new PrintStream(new File(filename));
            tree.save(out);
            out.close();
        } catch (FileNotFoundException e) {
            System.out.println("Error: " + e.getMessage());
        }
    }
}
}

```

```

public class QuestionNode {

    //Question stored in this node.
    public String question;

    //Reference to left subtree (if answer is yes).
    public QuestionNode left;

    //Reference to right subtree (if answer is no).
    public QuestionNode right;

    //Constructs a leaf node with the given question.
    public QuestionNode(String question) {
        this(question, null, null);
    }

    //Constructs a branch node with the given data and links.
    public QuestionNode(String question, QuestionNode left, QuestionNode right) {

```

```

        this.question = question;
        this.left = left;
        this.right = right;
    }
}

```

```

public class QuestionTree {

```

```

    private QuestionNode root;

```

```

    private Scanner input;

```

```

    private PrintStream output;

```

```

    private int totGames;

```

```

    private int gamesWon;

```

```

    /**

```

```

     * Constructor to initialize the new question tree.

```

```

     * @param input User interface for input.

```

```

     * @param output User interface for output.

```

```

     */

```

```

    public QuestionTree(Scanner input, PrintStream output) {

```

```

        //Throws an IllegalArgumentException if the passed

```

```

        //parameter is null.

```

```

        if (input == null || output == null)

```

```

            throw new IllegalArgumentException();

```

```

        this.input = input;

```

```

        this.output = output;

```

```

        root = new QuestionNode("A:Jedi");

```

```

    }

```

```

    /**

```

```

     * Plays one complete guessing game with the user.

```

```

     */

```

```

    public void play() {

```

```

        root = play(root);

```

```

    }

```

```

    /**

```

```

     * Private helper method to use recursion for play.

```

```

     * @param node The node to be returned.

```

```

     * @return the changed QuestionNode.

```

```

*/
private QuestionNode play(QuestionNode node) {
    //Tests to see if the node is an answer node.
    if (node.question.startsWith("A:")) {
        //Prints out the answer node data in a formatted manner to the desired PrintStream.
        output.print("Would your object happen to be " + node.question.substring(2) + "? ");
        //If the guess was correct, prints out an appropriate message and increments
gamesWon.
        if (UserInterface.nextAnswer(input)) {
            output.println("I win!");
            gamesWon++;
        } else //If the guess was incorrect, prompts the user for more information to alter the tree.
        {
            output.print("I lose. What is your object? ");
            String object = "A:" + input.nextLine();
            output.print("Type a yes/no question to distinguish your item from " +
node.question.substring(2) + ": ");
            String newQuestion = "Q:" + input.nextLine();
            output.print("And what is the answer for your object? ");
            //Appropriately adjusts the tree based on if the additional object should be added to
the left
            //or right of the QuestionNode.
            if (UserInterface.nextAnswer(input))
                node = new QuestionNode(newQuestion, new QuestionNode(object), node);
            else
                node = new QuestionNode(newQuestion, node, new QuestionNode(object));
        }
        //Increments the total games played regardless of whether the computer wins.
        totGames++;
    } else //If the node is a question node, then prints out the question and recursively calls
    //the method based on if the user enters yes (traverse down left tree) or no (traverse
    //down right tree).
    {
        output.print(node.question.substring(2) + " ");
        String answer = input.nextLine();
        if (answer.startsWith("y"))
            node.left = play(node.left);
        else
            node.right = play(node.right);
    }
    //Returns the node.
    return node;
}

```

```

/**
 * Saves the current tree in an output file.
 * @param output A PrintStream object representing the output file.
 */
public void save(PrintStream output) {
    //If the passed parameter is null, throws an exception.
    if (output == null)
        throw new IllegalArgumentException();
    //Calls on the private helper method to traverse the tree.
    save(output, root);
}

/**
 * A private helper method to traverse through the tree while saving
 * the information into an output file.
 * @param output The PrintStream object representing the output file.
 * @param node The current node to be examined.
 */
private void save(PrintStream output, QuestionNode node) {
    //Implicit base case to do nothing if the node is null.
    if (node != null) {
        //Writes the question in the node to the file (preorder).
        output.println(node.question);
        //Traverse down the left side.
        save(output, node.left);
        //Traverse down the right side.
        save(output, node.right);
    }
}

/**
 * Replaces the current tree by reading another tree from a file.
 * Precondition: The file exists and is in proper standard format.
 * @param input The Scanner that reads from a file.
 */
public void load(Scanner input) {
    //If the passed parameter is null, then throws an exception.
    if (input == null)
        throw new IllegalArgumentException();
    //If the file is empty, then does not load in any information.
    if (input.hasNextLine()) {
        //Initializes the root equal to the first line in the file.
        root = new QuestionNode(input.nextLine());
        root.left = add(root.left, input);
    }
}

```

```

        root.right = add(root.right, input);
    }
}

/**
 * Private helper method to add the nodes in the appropriate position.
 * @param node The node to be examined and returned.
 * @param input The user interface for input.
 * @return the changed QuestionNode
 */
private QuestionNode add(QuestionNode node, Scanner input) {
    //Returns null if the Scanner does not data left.
    if (!input.hasNext())
        node = null;
    else {
        //Stores the question in a variable for use later on.
        String question = input.nextLine();
        //If the passed node is null, then creates a new QuestionNode.
        if (node == null) {
            node = new QuestionNode(question);
            //If the line is a question, then calls on the method for both
            //the left and right side to build the tree.
            if (question.startsWith("Q")) {
                node.left = add(node.left, input);
                node.right = add(node.right, input);
            }
        }
    }
    return node;
}

//Returns the total games played so far.
public int totalGames() {
    return totGames;
}

//Returns the games won by the computer so far.
public int gamesWon() {
    return gamesWon;
}
}

public class UserInterface {

```

```
/**
 * Waits for the user to answer a yes/no question on the console and returns the user's
response
 * as a boolean (true for anything that starts with "y" or "Y").
 */
public static boolean nextAnswer(Scanner input) {
    String answer = input.nextLine();
    return answer.trim().toLowerCase().startsWith("y");
}
}
```